# beCook — auditoría integral y roadmap

**Fecha:** 15 de septiembre de 2026. **Código:** `feb0a7a` (10-09-2026). **Carácter:** auditoría, sin implementación ni despliegue.

## Alcance y grado de certeza

Se inspeccionaron rutas, componentes, librerías, configuración, lockfile, scripts, tests, migraciones y evidencia histórica de IA. Se consultaron el esquema y las políticas **reales** del proyecto Supabase beCook, sus asesores de seguridad y recuentos agregados. Se ejecutó una comprobación de lectura bajo RLS en una transacción `READ ONLY` con `ROLLBACK`.

Se probaron las pantallas en una compilación local de producción, con datos sintéticos, a **390 × 844**, **320 × 568** y, para creación de menú, **1440 × 900**. Son tamaños de navegador Chromium: **no equivalen a una prueba en Safari/iPhone físico**, teclado iOS o VoiceOver. Se revisaron login y alta, pero no se creó una cuenta ni se hizo login real. No se ejercitaron escrituras en producción, pagos OpenAI, despliegues ni migraciones.

Distingo tres clases de evidencia: **confirmado** por código/SQL/reproducción; **riesgo** deducido de un camino ejecutable, sin provocar pérdida real; **pendiente** cuando exige sesión, dispositivo o medición de campo. Las puntuaciones son juicio técnico, no un benchmark.

La carpeta original `menu_nutri/` permanece intacta y sin seguimiento. Solo se añaden esta documentación, el resultado de `npm audit` y un script de reproducciones de auditoría. No se modifica código de la aplicación. Las pruebas de navegador usaron una copia sin credenciales en `/private/tmp/becook-audit-20260915`, con un HTML auxiliar que carga datos sintéticos en su propio origen local.

## Mapa del sistema actual — antes del diagnóstico

### Repositorio y responsabilidades

| Capa | Implementación real |
|---|---|
| Rutas | `src/app`: App Router; layout raíz, páginas, redirecciones y tres API POST. |
| Pantallas cliente | `src/components/pages`: creación/importación, compra, despensa. Perfil, semana, login y alta permanecen directamente en `src/app`. |
| UI | `components/ui`, `components/layout`, `components/features`; adaptadores sobre algunos componentes shadcn. `DayCard.tsx` antiguo sin consumidores. |
| Dominio y conversión | `lib/importMenu.ts`, `isoWeek.ts`, `shoppingList.ts`; los tipos de semana se importan de `data/mockWeek.ts`. |
| Estado y caché | `useState`, `useEffect`, `useMemo`, `useRef`; cuatro familias de almacenamiento local. Sin store global, proveedor de sesión compartido ni hooks de dominio propios. |
| Supabase | Singleton `lib/supabase/client.ts`; Auth y seis módulos de acceso a datos desde el navegador. RLS determina qué filas pueden operar. No hay un backend propio de persistencia. |
| IA | `api/generate-menu` → `menu-generation/{prompt,runtime,validation}`; importación y compra mantienen lógica OpenAI dentro de sus handlers. |

| Capa | Implementación real |
|---|---|
| Datos de referencia | `nutritionist-recipes.json`: 27 nombres con procedencia, usados como ejemplos. `mockRecipes.ts`: ingredientes de demostración sin integración activa. |
| Configuración | Next 16.2.1, React 19.2.4, TypeScript estricto, Tailwind 4, React Compiler; dev con Webpack, build con Turbopack. Sin configuración Vercel versionada que permita verificar límites remotos. |
| Calidad | 7 tests `node:test`, loader TypeScript propio, scripts manuales Supabase, benchmark IA y documentación histórica. No hay CI versionada ni infraestructura frontend/E2E. |

**Rutas completas**

| Ruta | Tipo y función |
|---|---|
| `/` | Server Component que redirige a `/week`. |
| `/week` | Client Component dentro de Suspense: calendario ISO, carga local/remota, edición manual. |
| `/create-menu` | Wrapper servidor con Suspense y pantalla cliente; generar o importar, revisar y aplicar. |
| `/profile` | Cliente: perfil, preferencias, restricciones, cuenta y guardado. |
| `/login`, `/signup` | Cliente: Supabase Auth con email/contraseña y opción local. |
| `/shopping/list` | Reexporta pantalla cliente de compra. |
| `/shopping/pantry` | Reexporta pantalla cliente de despensa. |
| `/import` | Redirige a `/create-menu`; no preselecciona la pestaña importar. |
| `/shopping-list`, `/pantry` | Redirecciones de compatibilidad. |
| `/api/generate-menu` | POST Node: contexto → OpenAI → validación → propuesta. |
| `/api/import-menu` | POST Node: texto → OpenAI → parseo y complemento local de días. |
| `/api/generate-shopping-list` | POST Node: títulos de platos → OpenAI → ingredientes estimados. |
| Recetas | **No existe ruta, pantalla de receta ni ficha vinculada al menú.** |

El layout servidor añade fuentes y metadatos; la navegación inferior es cliente. Las pantallas se prerenderizan, pero sus datos de usuario se recuperan después del montaje. No existen Server Actions ni lógica de negocio en Server Components. Usar Client Components no elimina por sí mismo el prerenderizado.

**Flujos reales**

```
Entrada → /week → caché local por año/semana
                → Auth.getUser → Supabase weeks → week_days → UI

Login/alta → Supabase Auth → sesión del SDK en navegador → /profile
Perfil → localStorage → upsert profiles por user_id → feedback

Crear menú
  → perfil local (o remoto solo si no existe local) + despensa local
  → Auth.getUser → semanas recientes → week_days → INSERT petición
```

```
  → POST /api/generate-menu [sin autenticación de servidor]
  → prompt + 27 ejemplos filtrados → OpenAI Sol / fallback Terra
  → dos arrays de 7 nombres → validador → construir días → respuesta
  → navegador actualiza estado/telemetría de petición → vista previa

Usar menú
  → comprobar semana existente / confirmar reemplazo
  → caché local + guardar weeks/week_days ─┐
  → OpenAI genera compra + caché local ────┤ en paralelo parcialmente
  → INSERT shopping_lists + shopping_items
  → marcar petición aplicada → mensaje de éxito → /week

Compra
  → semana local seleccionada → caché por JSON de semana
  → si no coincide: OpenAI → caché → guardar contra última semana creada
  → marcar comprado → estado local + guardar OTRA lista completa

Despensa
  → lectura local → cambios locales → borrar conjunto remoto + insertarlo
  → nombres activos ayudan al prompt y añaden etiqueta «En despensa»
```

**Persistencia real**

- **7 tablas públicas con RLS activo:** `profiles`, `weeks`, `week_days`, `shopping_lists`, `shopping_items`, `pantry_items`, `menu_generation_requests`.
- Perfil: unicidad por `user_id`. Semana: índice único parcial por `(user_id, iso_year, iso_week)`. Los hijos autorizan mediante su padre y propietario.
- Claves locales: `becook-profile`, `becook-pantry-base`, `becook-week:YYYY-Www`, `becook-selected-iso-week`, `becook-shopping-list-cache`. **Ninguna incorpora usuario**. Solo la semana dispone de varias entradas; la compra tiene una única caché.
- No hay raciones/ingredientes/versiones de receta dentro de los slots: persiste principalmente el nombre del plato.

# 1. Resumen ejecutivo

1. **La prioridad es recuperar confianza en guardado, identidad de usuario y compra.** Añadir más IA o rediseñar la aplicación tiene menos retorno inmediato.
2. **Las rutas de IA permiten gastar sin sesión ni cuota en el código.** El modo invitado es intencional, pero necesita una política explícita de acceso al recurso de pago.
3. **Hay códigos de recuperación de Vercel en Git.** Su vigencia no está comprobada; deben considerarse potencialmente expuestos y sustituirse fuera de esta auditoría.
4. **RLS está razonablemente bien implementado.** El riesgo multiusuario más claro está en las cachés locales compartidas, no en una política general `true` de Supabase.
5. **La compra no es coherente entre semanas/dispositivos.** Usa la última semana creada al sincronizar, crea snapshots por cada clic y nunca lee la lista remota en la pantalla.
6. **El perfil y la base de datos no comparten contrato.** Hay fallos concretos con entrenamiento, objetivo «Otro» y alimentos rechazados.
7. **Elegir seis personas no hace que la compra sea para seis.** Compra presupone dos y pierde también el contexto de alergias/restricciones.
8. **La generación de menú ya fue simplificada acertadamente.** Tiene salida estructurada, inferencia acotada y tests; no aconsejo sustituirla por agentes/RAG.
9. **Las salsas vagas pasan la validación.** Se reproducen los cuatro ejemplos. La solución inmediata combina una regla de producto explícita, filtros y tests, conservando recetas reconocibles.
10. **La base visual es aprovechable y el build limpio pasa.** Faltan tests de los flujos de persistencia, parches selectivos de dependencias y mediciones reales de Core Web Vitals.

## 2. Estado general

| Área | /10 | Motivo |
|---|---|---|
| Arquitectura | 6 | Monolito adecuado al tamaño y módulos reconocibles; duplicación de fuentes de verdad y orquestación extensa en páginas. |
| Backend | 4 | Generador bien acotado; compra/importación y escrituras compuestas carecen de garantías equivalentes. |
| Frontend | 6 | UI reutilizable y TypeScript estricto; pantallas grandes, errores de sincronización y cargas inconsistentes. |
| UX | 5 | Flujo generar → revisar → aplicar claro; promesas de cantidades/sincronización y contexto de semana poco fiables. |
| Mobile | 6 | Filas táctiles amplias, safe-area y menú legible a 390 px; truncamientos a 320 px y perfil largo. Safari pendiente. |
| Seguridad | 4 | Buen aislamiento RLS; endpoints de pago abiertos, credenciales versionadas y dependencias pendientes de parche. |
| IA | 6 | Menú aproximadamente 7/10; cadena menú→compra aproximadamente 4/10 por pérdida de restricciones/ingredientes. |
| Testing | 3 | Siete pruebas útiles concentradas en IA. Sin protección automática del resto de recorridos críticos. |
| Performance | 5 | Build pequeño para un prototipo; llamadas y escrituras evitables. Sin datos de campo de LCP/CLS/INP. Provisional. |
| Mantenibilidad | 5 | Separación parcial; contratos manuales, compatibilidad antigua y falta de esquema reproducible/documentación operativa. |

# 3. Top 10 problemas

**Esfuerzo:** S = hasta media jornada; M = aproximadamente 1–2 jornadas; L = varias jornadas con diseño y validación. Se separan severidad del defecto y riesgo de modificarlo. Ordeno primero las contenciones sencillas y luego los trabajos que necesitan más garantías; no es una orden de reescribir todo.

## H1 — Códigos de recuperación versionados

- **Categoría:** BUG de seguridad. **Severidad:** alta; crítica si siguen vigentes. **Impacto:** posible acceso a la cuenta de infraestructura. **Esfuerzo:** S. **Riesgo del cambio:** bajo con acceso alternativo verificado.
- **Evidencia:** `vercel/recovery-codes.txt` está seguido por Git; contiene seis líneas con formato de código. Hay una entrada de historial de julio. No se intentó usar los códigos ni se verificó la visibilidad del repositorio remoto.
- **Recomendación:** regenerarlos/revocarlos, conservar los nuevos en un gestor de secretos y retirar el archivo del seguimiento futuro. Añadir regla de exclusión y escaneo. Borrar solo el archivo no invalida copias anteriores. No propongo reescribir historia como primer paso y no se ha hecho ninguna de estas acciones.

## H2 — Acceso no controlado a las tres rutas OpenAI

- **Categoría:** BUG de seguridad. **Severidad:** alta. **Impacto:** gasto, saturación y generación sin identidad fiable. **Esfuerzo:** S–M. **Riesgo del cambio:** medio: afecta a generación invitada.
- **Archivos:** generar menú, importar, compra.
- **Qué ocurre:** ninguno verifica token/sesión, ownership o cuota. Los fetch del cliente solo incluyen Content-Type. Los tests actuales incluso obtienen 200 con una petición sin sesión y OpenAI simulado. Supabase RLS no protege estas llamadas.
- **Recomendación:** exigir sesión verificada en servidor para las operaciones de pago; preservar lectura y edición manual local. Añadir cuota persistente y límite de concurrencia por usuario, más protección de IP. Si se decide mantener IA invitada, implementar presupuesto de invitado explícito; no basta un contador en memoria de una función Vercel. Limitar tamaño del cuerpo antes de procesarlo. No se ha comprobado si Vercel aporta reglas externas adicionales.

## H3 — Cachés mezcladas entre cuentas y perfil remoto que no se refresca

- **Categoría:** BUG. **Severidad:** alta. **Impacto:** datos anteriores visibles y restricciones de otra persona aplicadas al menú. **Esfuerzo:** M. **Riesgo del cambio:** medio por datos locales antiguos.
- **Archivos:** perfil: retorno temprano de caché, crear: misma estrategia, semana local, logout.
- **Qué ocurre:** cerrar sesión no limpia ni cambia ámbito. El usuario B puede seguir viendo el perfil de A; el perfil local impide consultar el remoto y puede acabar guardándose como perfil de B. Incluso con una sola cuenta, cambios de otro dispositivo no se recuperan si hay perfil local.
- **Recomendación:** resolver identidad antes de cargar datos, separar `guest` y `userId`, revalidar el perfil remoto y controlar cambios de sesión. Las entradas antiguas no tienen propietario verificable: conservarlas como datos locales pendientes de revisión, sin adjudicarlas automáticamente a una cuenta. Añadir prueba A → logout → B.

## H4 — Lockfile sin parches de seguridad recientes

- **Categoría:** DEUDA TÉCNICA. **Severidad:** alta por la superficie runtime; npm clasifica Next como crítica por avisos condicionados. **Impacto:** exposición y mantenimiento. **Esfuerzo:** S–M. **Riesgo del cambio:** medio.
- **Archivos:** package.json, resultado íntegro.
- **Medición:** 12 paquetes señalados: 1 crítico, 6 altos, 2 moderados y 3 bajos. **No son 12 vulnerabilidades explotadas ni 12 fallos propios.** Next está fijado en 16.2.1; npm propone 16.3.5.
- **Recomendación:** PR de mantenimiento acotada a Next/eslint-config-next y transitivas necesarias, seguido de Supabase Auth y herramientas de build afectadas. Verificar compatibilidad leyendo las guías de la versión objetivo. No usar `npm audit fix --force` indiscriminadamente.

- La publicación oficial 16.3.5 y los parches de agosto respaldan actualizar. El RCE de Windows no corresponde a esta aplicación App Router/macOS/Vercel; el de AVIF requiere imágenes controladas por atacante y no se observó ese flujo. También existen avisos RSC anteriores al parche 16.2.5; no se realizaron pruebas ofensivas para establecer explotabilidad aquí.

## H5 — Contrato de perfil incompatible con Supabase

- **Categoría:** BUG. **Severidad:** alta funcional. **Impacto:** preferencias perdidas o imposibles de sincronizar. **Esfuerzo:** S–M. **Riesgo del cambio:** bajo con adaptación compatible.
- **Archivo:** mapeo de perfil. **Evidencia adicional:** columnas y CHECK reales consultados.
- UI/TS permiten entrenamiento **0–7**, pero `profiles_weekly_training_days_check` solo admite **0–5**.
- Si el primer objetivo es `other`, `getLegacyGoalForStorage` devuelve `other`, pero `profiles_goal_check` no lo admite; el array `goals` sí lo admite.
- `disliked_foods` es **text** en la base real, pero el código lo escribe/tipa como `string[]`. Al leer texto, `parseStringArray` devuelve `[]`: desaparecen los rechazos. El resultado exacto del intento de escritura depende de la conversión de PostgREST; el fallo de lectura está determinado por el código.
- **Recomendación:** adaptador explícito texto↔lista para el esquema actual, valor compatible en el campo legacy sin perder `goals`, y ampliación no destructiva del CHECK a 7 cuando se autorice implementar. Test del contrato real. No eliminar columnas ni migrar datos a ciegas.

## H6 — Compra pierde raciones y restricciones del menú

- **Categoría:** BUG. **Severidad:** alta. **Impacto:** cantidades incorrectas y posibles ingredientes incompatibles. **Esfuerzo:** M. **Riesgo del cambio:** medio.
- **Archivos:** contrato de compra, prompt de compra, perfil promete cantidades.
- **Qué ocurre:** la API recibe únicamente week, asume dos personas e infiere ingredientes a partir de títulos. El perfil/restricciones no llegan. El menú tampoco conserva ingredientes ni raciones. Una salsa puede introducir un alérgeno que no aparece en el nombre.
- **Recomendación:** persistir contexto por semana —raciones y snapshot/versionado de restricciones— y transportarlo hasta compra. A corto plazo corregir la promesa de cálculo y evitar afirmar compatibilidad certificada. A medio plazo generar/reutilizar una misma ficha de ingredientes; validar compatibilidad antes de mostrar la compra. No resolverlo añadiendo únicamente otra frase al prompt.

## H7 — Compra vinculada a semana incorrecta, duplicada y no recuperada

- **Categoría:** BUG. **Severidad:** alta. **Impacto:** compra inconsistente y coste/almacenamiento crecientes. **Esfuerzo:** M. **Riesgo del cambio:** medio.
- **Archivos:** fallback a última semana, cada clic, INSERT de lista completa.
- **Qué ocurre:** la pantalla no pasa `weekId`; se elige por `created_at`, no por semana seleccionada. Cada marcado inserta una lista y todos sus ingredientes. Hay función de lectura remota, pero la pantalla nunca la usa. En otro dispositivo puede no aparecer nada o regenerarse con todos los checks vacíos.
- **Evidencia real agregada:** 28 semanas, 108 listas, 3.427 ingredientes; una semana tiene 42 listas. Confirma multiplicidad, aunque no permite atribuir cada fila a un clic concreto.
- **Recomendación:** identidad obligatoria de semana/lista, recuperar primero la lista remota y actualizar solo el ingrediente marcado. Mantener IDs estables, ordenar escrituras concurrentes y mostrar error/reintento. Conservar los snapshots antiguos hasta decidir su política; no borrarlos en la reparación inicial.

## H8 — Reemplazos no atómicos de semana y despensa

- **Categoría:** BUG. **Severidad:** alta. **Impacto:** pérdida parcial o sobrescrituras durante fallos/concurrencia. **Esfuerzo:** M. **Riesgo del cambio:** medio.
- **Archivos:** delete→insert de días, reemplazo de despensa, editar semana sin esperar.

- **Qué ocurre:** se borra y se inserta en peticiones separadas. Si la segunda falla, falta el conjunto. Dos cambios rápidos pueden intercalar delete/insert o terminar fuera de orden. Crear semana/lista también puede dejar un padre sin hijos.
- **Recomendación:** actualización por slot/item para cambios pequeños y operación transaccional para aplicar/reemplazar el conjunto; ownership, versión esperada e idempotencia. No necesita microservicios. **Riesgo reproducible por lectura de secuencia; no se provocó pérdida real.** Actualmente las 28 semanas tienen siete días.

### H9 — Éxito mostrado aunque no se haya sincronizado

- **Categoría:** BUG. **Severidad:** media-alta. **Impacto:** falsa confianza y pérdida al cambiar de dispositivo. **Esfuerzo:** S. **Riesgo del cambio:** bajo.
- **Archivos:** guardado devuelve null, aplicación muestra éxito.
- **Qué ocurre:** se escribe localmente, el helper captura el fallo remoto y devuelve `null`; la página ignora este resultado y anuncia actualización correcta. El fallo de compra se reduce a warning. El perfil ya distingue mejor local/remoto/error y sirve como referencia.
- **Recomendación:** resultado explícito `{local, remote, shopping, error}`; mensajes distintos para invitado, sincronización pendiente y fallo. No volver a facturar IA para reintentar una escritura. Habilitar acceso al menú tras guardarlo sin esperar una generación de compra potencialmente larga.

### H10 — Política culinaria de salsas no aplicada

- **Categoría:** MEJORA DE PRODUCTO. **Severidad:** media. **Impacto:** platos poco útiles, compra ambigua y menor confianza. **Esfuerzo:** S para primera barrera; M para recuperación selectiva. **Riesgo del cambio:** bajo–medio por falsos rechazos.
- **Archivos:** prompt, validación.
- **Confirmación:** `selectionToMenu` acepta «Merluza en salsa verde», «Pollo con salsa especial», «Pavo con salsa casera» y «Bacalao con salsa mediterránea».
- **Recomendación:** regla explícita de seleccionar otro plato cuando la salsa sea ambigua, catálogo de nombres concretos y tests negativos/positivos. No “arreglar” el título inventando ingredientes. Ver sección IA para cómo sustituir el plato con seguridad y coste acotado.

## 4. Quick wins

Orden sugerido, dentro de cada paquete de menor esfuerzo/riesgo:

| Cambio | Razón concreta | Estimación |
|---|---|---|
| Señalar y retirar de uso los códigos expuestos | H1; control de infraestructura | <1 h de operación, fuera de esta auditoría |
| Corregir fechas y firma de caché | Fechas fijas provocan información falsa y una segunda compra IA | 1–2 h + pruebas |
| Resultado local/remoto explícito | H9; evita éxitos engañosos | 1–2 h |
| Regla contra salsas vagas + tests | H10; fallo confirmado | 1–2 h para barrera inicial |
| Adaptar `goal` legacy y lectura/escritura de rechazos | H5; contrato conocido, sin rediseñar esquema | 1–3 h |
| Mejorar contraste y foco de chips | Texto blanco/naranja: 3,20:1; foco oculto en `sr-only` | 1–2 h |
| Etiqueta de semana en compra y texto «Crear/reemplazar menú» | «Editar menú» abre un formulario vacío de generación | <1 h |

Son estimaciones por cambio, **no promesa de incluir toda la tabla en una sesión**. La recuperación remota, la atomicidad y las cuotas persistentes no son quick wins cosméticos.

# 5. Roadmap fase 1 — mejoras rápidas y seguras

**Objetivo:** contener riesgos inmediatos y reparar errores visibles sin reestructurar persistencia. **Duración:** 1–2 sesiones, aproximadamente 8–12 horas; recortar tareas opcionales si las dependencias requieren ajustes.

### Sesión 1: protección mínima y comportamiento veraz

1. Preparar la retirada del archivo de recuperación y su exclusión. La regeneración de credenciales se hace en la cuenta correspondiente, con acceso alternativo, cuando se autorice ejecutar esa operación.
2. Añadir helper de verificación de sesión a los tres handlers y token en sus consumidores. Propuesta de producto: pedir cuenta para IA; permitir consultar y editar datos locales. Actualizar mensajes de invitado. Límite de tamaño para texto, semana, slots y cuerpo HTTP.
3. Sustituir `null` ambiguo por resultados de guardado claros en semana/aplicación; no anunciar sincronización cuando falla y no perder la propuesta al fallar compra.
4. Abrir un cambio acotado de Next/eslint-config-next a la versión parcheada verificada, con lockfile y validaciones. No añadir funcionalidades nuevas de Next.

### Sesión 2: calendario, caché y calidad culinaria

1. Eliminar `previewDatesByDay`; derivar fechas de la identidad ISO y del **nombre/posición real de cada día**, también para importaciones parciales.
2. Hacer que la firma de compra use contenido y contexto canónicos, nunca el texto de fecha presentado. Comprobar que aplicar → abrir semana → abrir compra reutiliza el resultado.
3. Sincronizar anterior/siguiente/hoy con parámetros de URL; validar semana 53 contra su año.
4. Añadir regla de salsas vagas al prompt y validador. Si todavía no existe sustitución segura, rechazar con mensaje concreto; no aprobar el plato por renombrarlo. La sustitución selectiva llega en fase 2.
5. Si queda capacidad: adaptar los dos campos legacy de perfil, corregir contraste del CTA y foco visible en chips. El resto queda explícitamente para fase 2.

**Criterios de salida:** peticiones de IA no autorizadas rechazadas antes del mock OpenAI; error de guardado no produce mensaje de sincronizado; fechas correctas en semana completa/parcial y cambio de año; firma idéntica antes/después de navegar; cuatro salsas vagas rechazadas y salsas concretas aceptadas; tests, lint, tipos y build limpios.

**Importante:** fase 1 no convierte aún beCook en un producto listo para apertura pública. Separación de cachés, cuotas durables y persistencia coherente siguen siendo requisitos de fase 2 antes de ampliar usuarios.

# 6. Roadmap fase 2 — estructura con impacto medio

**Objetivo:** multiusuario y compra fiables. Estimación orientativa: 5–8 sesiones, separadas en entregas revisables.

| Orden | Paquete | Cambio concreto y condición de aceptación |
|---|---|---|
| 1 | Identidad y caché | Ámbitos guest/user, gestión de cambio de sesión, revalidación de perfil, importación local explícita. A no ve datos de B; un segundo dispositivo recupera perfil/semana/compra/despensa. |
| 2 | Contrato de perfil | Adaptadores reales; CHECK 0–7 mediante cambio no destructivo; tipos derivados del esquema y fixtures de contrato. «Otro», seis/siete días y rechazos sobreviven a guardado/lectura. |
| 3 | Compra por semana | Pasar `weekId`, `listId` e `itemId`; lectura remota; PATCH de un check, versión de menú y raciones. Marcar un producto no crea una lista nueva ni modifica otra semana. |
| 4 | Escrituras seguras | Aplicar menú en transacción, editar un slot, modificar un artículo de despensa. Versiones/concurrencia e idempotencia; falla una operación y el conjunto anterior permanece consistente. |
| 5 | Contexto culinario | Raciones y restricciones vinculadas a semana/ingredientes. Filtro sobre compra. Una propuesta para seis conserva seis en la compra y en otro dispositivo. |
| 6 | Robustez de IA | Timeouts, retries y errores explícitos también en compra/importación; cuotas persistentes, límite de concurrencia y telemetría servidor por intento/flujo. Cancelar el navegador no deja actividad sin límite. |
| 7 | UX de recuperación | Borrador recuperable, reintentar sincronización sin regenerar, error de compra con botón de reintento, resumen de restricciones antes de generar y perfil cargado antes de habilitar esa acción. |
| 8 | Calidad y medición | Tests de componentes y contratos, CI mínima, métricas de campo Web Vitals; ampliar dataset culinario incluyendo salsas/alergias. |

Para sustituir un plato ambiguo: elegir una alternativa validada compatible cuando exista; en otro caso, una única reparación acotada del slot, con validación completa del menú después. Registrar el motivo y el coste. No introducir bucles ocultos ni regenerar toda la semana por defecto.

# 7. Roadmap fase 3 — arquitectura y producto a largo plazo

1. **Ficha de receta compartida:** identidad estable, nombre, ingredientes, raciones y preparación. Catálogo pequeño, revisado y útil; ampliar por demanda. El menú referencia una versión concreta y la compra agrega sus ingredientes. Reemplazo progresivo de la inferencia independiente de compra.
2. **Biblioteca híbrida:** aprovechar las 27 referencias actuales, pero completar ingredientes/alérgenos antes de convertirlas en recetas garantizadas. Generación libre como vía controlada, sin RAG, embeddings o agentes mientras no exista un problema de recuperación que lo justifique.
3. **Simplificar perfil:** empezar por comensales, restricciones y preferencias. Posponer/ocultar medidas corporales que hoy no usa el generador. Hacerlo tras resolver sus campos obligatorios en DB; no eliminar la obligatoriedad solo en HTML y romper la persistencia.
4. **Sustituir platos desde la semana:** acción reconocible con alternativas y efecto visible en compra, manteniendo edición manual. Actualmente solo se cambia texto.
5. **Despensa proporcionada al valor:** mantener «habitualmente tengo», con omisión manual en compra. Cantidades/caducidad solo si se observa uso real; no construir un inventario doméstico completo por anticipación.
6. **Reducir pantallas monolíticas:** extraer hooks/controladores al estabilizar los contratos y mover lecturas iniciales al servidor solo donde mejoren tiempo visible y sesión. Monolito Next + Supabase sigue siendo suficiente.

**Puerta de entrada:** fase 2 fiable y métricas reales. No priorizar PWA, microservicios, colas de agentes, índices masivos o rediseño estético.

# 8. Frontend, UX, mobile y diseño — por pantalla

| Pantalla | Problema real | Recomendación |
|---|---|---|
| Login/alta | Explica almacenamiento local, pero poco el valor menú→compra. Alta menciona «Supabase». Login siempre lleva a perfil. No existe recuperación de contraseña/callback dedicado. | Texto centrado en guardar planes entre dispositivos; volver al destino deseado. Añadir recuperación/confirmación probada. No bloquear lectura local. |
| Perfil | Datos personales aparecen antes de restricciones; nacimiento requerido, peso/altura/sexo no llegan al prompt actual. Defaults hombre/175/75 pueden parecer información real. 900 líneas; mezcla cuenta, datos y preferencias. | Restricciones y comensales primero; campos avanzados secundarios. Corregir contrato antes de cambiar esquema. Indicar qué se conserva y qué afecta al resultado. |
| Creación | Petición semanal bien delimitada, pero no muestra restricciones activas. Puede generarse antes de recuperar el perfil remoto, con `profile=null`. Estilos y objetivos se solapan; «Equilibrado» significa 20–45 min en dificultad. | Resumen del perfil aplicado, bloquear mientras se carga, tiempo estimado visible por dificultad. Mantener texto semanal de 300 caracteres. Separar preferencias permanentes de excepción semanal. |

| Pantalla | Problema real | Recomendación |
|---|---|---|
| Vista previa | Fechas fijas, 14 platos no editables aquí; propuesta solo en memoria. Al regenerar no se limpia inmediatamente la propuesta anterior y «Usar» no depende de `isLoading`: se puede aplicar la anterior mientras llega otra. | Fechas correctas, vincular propuesta a su contexto, deshabilitar aplicación mientras cambia y conservar borrador recuperable. Añadir edición por slot cuando esté protegida la compra. |
| Semana | Comida/cena legibles; tocar plato permite texto libre, pero no hay sugerencias ni receta. «Editar menú» abre creación vacía. Flechas no actualizan URL. A 320 px se trunca «14–20 septiem…». | «Crear/reemplazar menú» para la acción global; hacer explícita edición del plato. URL como selección compartible, título adaptable y feedback de guardado por slot. |
| Compra | Buen marcado por fila y conteos; no indica semana/raciones. Solo muestra una categoría; «En despensa» no cambia lo que falta. Errores sin reintento; ausencia local parece ausencia total. | Mostrar semana y comensales; recuperar remoto; distinguir pendiente/comprado/ya tengo. Botón reintentar y acceso directo a semana en vacío. Mantener categorías, añadir vista «Todo/Pendiente» si reduce cambios de pestaña. |
| Recetas | La función descrita en el contexto no existe como recorrido de producto. El catálogo mock no tiene consumidores. | No anunciar recetas/cálculo de sus cantidades como capacidad actual. Construir ficha compartida con compra en fase 3, sin una sección de biblioteca aislada. |
| Despensa | Se cargan datos locales, no remotos; cada cambio reemplaza todo el conjunto remoto. Siete categorías visibles aun vacías; selección inicial «básicos» vacía con un solo aceite. Nombres largos truncados incluso a 390 px. | Recuperación y operaciones por artículo primero. Mostrar categorías útiles/permitir dos líneas; explicar «habitualmente tengo». Mantenerla opcional y evitar contabilidad de cantidades ficticia. |

**Mobile y accesibilidad: confirmado frente a pendiente**

- **Confirmado:** sin desbordamiento horizontal en semana/despensa comprobadas a 320 px; perfil sin overflow a 390 px. No implica que todos los estados/teclados estén cubiertos.
- **Confirmado:** filas de compra de al menos 64 px; el `<label>` completo activa el checkbox. Navegación inferior amplia, safe-area y reducción de animación presentes. Mantenerlos.
- **Confirmado:** steppers de perfil de **36 × 36 px** y chips de aproximadamente 32 px de alto; aumentar a unos 44 px en móvil mejoraría precisión. Esto es una recomendación de comodidad, no una afirmación de incumplimiento universal del mínimo WCAG.
- **Confirmado:** texto blanco sobre `#e0703c` tiene contraste **3,20:1**, insuficiente para texto normal según WCAG 1.4.3. Afecta CTAs y categorías seleccionadas. Corregir el token/uso; mantener la paleta.
- **Confirmado:** chips con inputs `sr-only` no trasladan visualmente su foco al label; `StateMessage` no anuncia error/éxito con región viva. Contenedores `tablist` de categorías contienen botones `aria-pressed`, no un patrón completo de tabs. Usar grupo de botones o implementar tabs completos; no solo cambiar un atributo.
- **Confirmado:** editor de plato tiene placeholder, pero no label asociado/aria-label propio. Añadir nombre contextual. Las pantallas deberían tener títulos de documento distintos para navegación y

orientación accesible.

- **Matiz importante:** las clases parecen declarar inputs pequeños, pero el CSS global `font: inherit` prevalece: en perfil se midieron selects de **16 px**. No hay evidencia para afirmar que esos campos provocan zoom iOS. Verificar Safari antes de atribuirle ese bug.
- **Pendiente:** teclado virtual, scroll con input enfocado, Safe Area real, orientación horizontal, VoiceOver y uso prolongado de compra en iPhone. Las barras sticky/fijas reducen el área útil; no se comprobó una acción permanentemente inaccesible.

**Diseño visual**

**UX:** fechas, identidad de semana, mensajes falsos, opciones poco explicadas. **Visual/accesibilidad:** contraste, foco y truncamiento. **Preferencia estética:** color cálido, esquinas redondas y cards; no recomiendo sustituirlos por gusto.

Hay tokens de color, radios y jerarquía tipográfica reutilizables. Los formularios de login/creación usan ancho máximo de 6xl frente a 3xl en semana/perfil/compra; a 1440 px se observa dispersión entre label y control. Limitar ancho de formulario aporta legibilidad. Evitar mezclar clases conflictivas de spacing mediante concatenación: `pageContentClass` aporta gaps grandes y luego se añaden otros. No emprender un nuevo design system.

# 9. Backend — por subsistema

| Subsistema | Evaluación y acción priorizada |
|---|---|
| Generación | Separación prompt/runtime/validación acertada. Falta auth/cuota; el perfil llega desde el navegador, no desde una identidad validada. Aplicar H2/H3 antes de confiar en el contexto. |
| Importación | Structured Outputs y parser local de recuperación de días. Siempre llama al LLM antes del parser, aun con tabla clara. Usar parseo determinista primero solo cuando sea inequívoco; fallback IA para texto ambiguo. |
| Validación de importación | Arrays/dishName sin límites fuertes; JSON inválido acaba en 500; no se comprueba `finish_reason`/refusal de forma explícita. Añadir estados 400/422/502 y límites; no aplicar validación culinaria para rechazar silenciosamente un menú que el usuario pegó deliberadamente. Advertir incompatibilidades. |
| Compra | Validador de entrada solo comprueba day/date y existencia de lunch/dinner; acepta slots de tipos arbitrarios. Sin máximo de días o texto. Schema de salida tiene estructura, pero admite strings vacíos que luego se descartan. Validar contratos completos. |
| Persistencia de semana | Índice único ISO útil, pero check→insert puede competir. `insertWeekRow` reintenta cualquier error sin vínculo de generación, aunque no sea un fallo de compatibilidad de esa columna. Limitar fallback o retirarlo al estabilizar esquema. |
| Semanas/calendario | `parseIsoWeek` acepta 2025-W53, que se convierte en 2026-W01. Importar solo miércoles y fechar por índice muestra lunes. Corregir calendario y tests de parciales, años y DST. |

| Subsistema | Evaluación y acción priorizada |
|---|---|
| Compra/caché | `JSON.stringify(week)` incorpora texto de fecha. Vista previa marzo/abril → semana real septiembre cambia firma sin cambiar platos: otra llamada IA. La firma tampoco incorpora raciones/restricciones/versionado, necesarios al arreglar H6. |
| Despensa | Se usa como preferencia secundaria y etiqueta, sin restar cantidades. Es honesto no restar un stock que no se conoce, pero el usuario necesita poder indicar «ya lo tengo». Recuperación remota hoy ausente. |
| Telemetría | El navegador crea/actualiza filas; un cierre puede dejar `generating`, y el usuario puede modificar su propia telemetría mediante RLS. Sirve como diagnóstico, no como contabilidad autorizada ni sistema de cuotas. Mover responsabilidad al servidor. |
| Errores | Generador registra métricas sin prompts; compra/importación aún registran respuesta inesperada o error completo. Normalizar logs con códigos y request ID, sin perfiles/platos/texto libre. |

**Ownership:** los filtros cliente son útiles, pero no sustituyen RLS. Las consultas «latest» sin filtro de usuario no prueban fuga remota con estas políticas. Sí son un mal contrato para identificar la semana. Algunas FK entre entidades con `user_id` independiente no comprueban que la referencia pertenezca al mismo usuario: endurecer al consolidar las operaciones, sin afirmar acceso cruzado demostrado.

# 10. IA — estado actual y mejora recomendada

### Configuración efectiva

| Flujo | Modelo/configuración observada | Límites y telemetría |
|---|---|---|
| Menú | `MENU_OPENAI_MODEL` o Sol; fallback Terra; `reasoning_effort: none`. No hay overrides MENU en el archivo local. | 1.200 tokens, 20 s/intento, máximo dos modelos distintos, SDK retries=0, `store:false`, señal de cancelación; maxDuration=45. Métricas y Server-Timing. |
| Importación | `OPENAI_MODEL`; archivo local = Sol. Chat Completions. | Sin reasoning explícito, sin límite de salida/timeout propio, SDK por defecto. Sin telemetría de coste del flujo. |
| Compra | `OPENAI_MODEL`; archivo local = Sol. Responses. | Sin reasoning explícito, sin límite de salida/timeout propio; no pasa la señal del cliente a OpenAI; no explicita `store:false`. Sin telemetría completa. |

**No confundir configuración declarada y usada:** `OPENAI_REASONING_EFFORT=high` existe localmente y se exporta, pero **ningún handler de compra/importación lo usa**. No es correcto atribuirles `high` por esa variable. Tampoco usan el fallback exportado. El SDK instalado declara por defecto dos retries

y timeout de diez minutos; un entorno serverless puede terminar antes. Unificar presupuestos por flujo, sin imponer automáticamente los mismos valores a todas las tareas.

### Lo que funciona

- Prioridades separadas: alergias/restricciones > petición semanal > preferencias.
- Salida compacta de 14 nombres, con días/estados construidos por código.
- Structured Outputs estricto, validación adicional de cardinalidad, duplicados normalizados y conflictos visibles.
- Contexto enviado al modelo sin IDs, nacimiento, altura ni peso; ejemplos con procedencia.
- Fallback solo ante conexión/timeout, 404/modelo ausente y 5xx. No multiplica llamadas ante 400, 429 o validación fallida.
- El coste por modelo desconocido se devuelve como desconocido (`null`), no como cero.

Structured Outputs asegura ajuste al esquema, no que una receta sea culinariamente concreta ni compatible con ingredientes ocultos. Mantener validación de dominio.

### Salsas vagas: diagnóstico por factor

| Factor | Diagnóstico |
|---|---|
| Prompt | Pide nombres concretos, pero no expresa la regla «si no puedes describir la salsa, selecciona otro plato» ni define los ejemplos prohibidos. |
| Modelo | No hay evidencia que justifique culpar/cambiar Sol. Cualquier modelo puede cumplir el schema y devolver «salsa verde». |
| Estructura | Solo nombres; no hay descriptor de salsa o ingredientes que permita verificar su significado. |
| Catálogo | Existe, pero es pequeño, solo de nombres y abierto: no restringe a identidades revisadas. |
| Validación | La lista negra se aplica a diez títulos genéricos exactos, no a salsas vagas dentro de un plato. Los cuatro ejemplos pasan. |

### Resolución propuesta:

1. Definir salsa concreta: un nombre reconocible y suficientemente descriptivo o sus ingredientes principales. «Salsa de tomate y albahaca» es clara; «especial/casera/mediterránea» no aporta composición. «Verde» requiere concretar, por ejemplo perejil y ajo, solo si la receta realmente utiliza esa preparación.
2. Instruir al modelo a elegir otro plato cuando no pueda cumplirlo. No añadir ingredientes supuestos al título de una propuesta ya elegida.
3. Detector acotado de estas fórmulas, pruebas con variaciones y lista de casos válidos; no prohibir toda aparición de «casera» en «pizza casera» ni todas las salsas reconocibles.
4. Como primera contención, devolver un error culinario concreto cuando falle. Para cumplir la sustitución sin hacer repetir todo al usuario, en fase 2 usar alternativa de catálogo validada o una reparación de slot con presupuesto máximo y revalidación.
5. Validar también duplicados, restricciones y petición semanal tras sustitución. Una alternativa no es válida solo por tener un nombre bonito.

### Restricciones y calidad culinaria

Los regex detectan algunos ingredientes **visibles**, no todos los alérgenos. `otherAllergies` se conserva en el prompt, pero el filtro no la interpreta: se comprobó que «Ensalada de sésamo» con alergia textual a sésamo no se bloquea determinísticamente. Los títulos tampoco permiten verificar cantidades, técnicas, tiempo real o frecuencias nutricionales. No prometer ausencia de alérgenos ni dieta terapéutica.

La acción útil es compartir ingredientes y restricciones hasta compra, ampliar casos de evaluación y permitir fallar de forma visible. Un clasificador LLM adicional puede ayudar a detectar problemas, pero no sustituye la información que falta ni justifica una garantía.

### Latencia, coste y tareas deterministas

La evidencia histórica del repositorio reporta tres casos finales: **7,38–8,86 s**, media **7,91 s**, y coste estimado medio **$0,008196 por menú**. Son mediciones anteriores del handler y OpenAI; **no las he repetido ni representan esta sesión, compra, navegador o percentil 95**. El propio documento explica que las versiones de prompt/modelo difieren. Fuente: benchmark documentado.

La consulta actual tiene 57 peticiones y dos siguen `generating`. Mezcla modelos y versiones sin `prompt_version`; no sirve para calcular la latencia actual exacta. Los costes almacenados son estimaciones históricas y faltan importación/compra e intentos sin usage. No presentarlos como factura.

Usaría código para: calendario, parser inequívoco de tabla, sumar ingredientes compatibles, normalizar unidades, checks, caché, deduplicación, restricciones sobre ingredientes conocidos y selección de alternativas revisadas. La elección creativa del menú y el texto ambiguo sí justifican LLM. El ahorro inmediato está en evitar compras duplicadas, no en reducir unos caracteres del prompt.

## 11. Seguridad — hallazgos relevantes

### Confirmado en Supabase real

- RLS habilitado en las siete tablas públicas. Políticas de SELECT/INSERT/UPDATE/DELETE limitadas a `authenticated`, con comparación de `auth.uid()` o recorrido hasta la semana propietaria. UPDATE incluye `WITH CHECK` en las políticas observadas.
- Prueba `READ ONLY` con rol authenticated y un propietario existente: un perfil propio visible; **0 perfiles, semanas o despensa ajenos**, y cero hijos visibles sin padre autorizado en las comprobaciones realizadas. No demuestra todas las escrituras ni un login real; sí aporta evidencia de aislamiento de lectura.
- No se encontró service role en el código cliente ni en los nombres de variables locales. La clave anon pública es esperable; no debe confundirse con una clave privada. El aislamiento depende de RLS.
- Escaneo de patrones sobre archivos seguidos no encontró claves OpenAI, JWT largos o claves privadas incrustadas. No es un análisis completo de todo el historial. El archivo de recuperación sigue siendo el hallazgo de credenciales relevante.

### Avisos del asesor, interpretados

| Aviso | Evaluación y prioridad |
|---|---|
| `set_updated_at` sin search_path fijo | Función invoker de trigger, cuerpo mínimo `new.updated_at = now()`. Endurecimiento pequeño; no indicio de fuga de datos. Remediación. |
| `rls_auto_enable` SECURITY DEFINER ejecutable por anon/authenticated | Es una **event trigger**, con search_path `pg_catalog`, para activar RLS al crear tablas. El aviso de privilegios no prueba que sea una RPC ordinaria explotable. Revisar/restringir EXECUTE según necesidad, sin romper el trigger. Remediación. |
| Protección de contraseñas filtradas desactivada | Activar si lo permite el plan y completar recuperación de cuenta. Documentación. |

**Orden de seguridad:** códigos y rutas de pago → cachés/identidad y parches → cuota/telemetría confiable → endurecimiento menor. No hay evidencia suficiente para afirmar compromiso, fuga remota entre usuarios o RCE explotado.

**Vercel:** se verificó el límite `maxDuration=45` del código de menú, no los overrides del despliegue, WAF, variables remotas, logs ni Web Vitals de producción. No se usaron los códigos encontrados para entrar. La auditoría del repositorio no certifica que el deployment remoto ejecute exactamente este commit.

# 12. Testing — qué existe y qué falta

### Existe

tests/menu-generation.test.cjs: **7 tests, 7 pasan**. Cubren transformación 7×2, outputs incompletos/duplicados/genéricos, conflictos visibles, construcción de contexto, cálculo de coste, transporte con fallback y errores de la ruta mediante OpenAI simulado.

Son pruebas de módulo/handler; no HTTP desplegado, login real, RLS de escritura ni UI. El loader transpila TS sin hacer typecheck; por eso tipos/build son comprobaciones separadas. Los tests de precio fijan la tabla local; que pasen no valida precios vigentes de la factura.

Los scripts `test-save-*` son comprobaciones manuales con potencial de escritura, no una suite aislada. Algunos insertan sin usuario y no reflejan RLS autenticado; no se ejecutaron. Los scripts de lectura pueden mostrar datos completos: para la auditoría se usaron agregados. No hay componentes testados, mocks frontend, E2E ni pipeline CI versionado.

### Estrategia proporcionada

| Prioridad | Prueba que protege un fallo real |
|---|---|
| P0 | Los tres handlers rechazan sesión ausente/inválida antes de llamar a OpenAI; cuota agotada no factura. |
| P0 | A → salir → B no hereda perfil, restricciones, semana o compra; no hay adopción silenciosa de invitado. |
| P0 | Guardar perfil con «Otro», 6/7 días y rechazos; lectura equivalente con el contrato SQL real. |
| P0 | Aplicar menú: fallo remoto, fallo de compra, doble clic, reintento de escritura sin regenerar. |
| P1 | Editar dos slots rápidamente y fallar un insert: consistencia/idempotencia en base de pruebas. |
| P1 | Semana anterior/siguiente/hoy y URL, cambio de año, semana 53 válida/inválida, importación parcial. |
| P1 | Compra de semana antigua modifica esa lista; un clic no inserta otra; recarga y segundo dispositivo conservan checks. |
| P1 | Raciones/restricciones llegan a compra; salsas vagas se rechazan o sustituyen por alternativa válida. |
| P1 | Componentes: loading de perfil, errores/reintento, controles de plato, foco/etiquetas y marcado optimista con fallo. |

Añadir React Testing Library con un runner compatible cuando se empiece fase 2, no por imponer porcentajes. Una DB de pruebas o rama aislada debe proteger contratos y RLS A/B; nunca apuntar la suite de escritura a producción por defecto. CI mínima: instalación desde lockfile, lint, tipos, tests y build. Posponer E2E nuevo; mantener una checklist manual de iPhone mientras se estabiliza la persistencia.

# 13. Performance — evidencia y límites

### Comprobaciones de esta sesión

El script de reproducciones verifica los casos de salsas vagas, alergia textual, firma de compra y calendario sin red ni escritura de datos. Se ejecutó correctamente desde la raíz con `node --require ./scripts/register-typescript.cjs docs/audit-2026-09-15/reproducciones.cjs`. Sus aserciones documentan los defectos actuales; deberán invertirse o adaptarse cuando se corrijan, antes de incorporarlas a una suite de regresión.

| Prueba | Resultado |
|---|---|
| `npm test` original | 7/7, aproximadamente 0,41 s en la ejecución observada. |
| `typecheck` original | Falla por `.next/types/routes.d 2.ts`: `Duplicate identifier 'LayoutProps'`. Artefacto generado duplicado, no error localizado en código fuente. |
| `lint` original | Dos intentos no terminaron ni devolvieron diagnósticos; se detuvieron. No se clasifica como fallo de lint del código. |
| Copia limpia + `npm ci` del lockfile | Lint y typecheck pasan. Sin actualizar versiones. |
| `npm run build` limpio | Pasa con Next 16.2.1/Turbopack; 17 páginas generadas; compilación 3,0 s, TypeScript 1,85 s. No sumar como si fuese toda la duración wall-clock. |
| Assets del build | 29 JS: 963,1 KiB sin comprimir / 292,4 KiB gzip calculado; CSS 56,3 / 9,8 KiB; 11 archivos de fuentes, 143,0 KiB. **Totales del build**, no descarga por página. |
| Scripts referenciados por HTML | Semana: 201,2 KiB gzip; creación: 205,9; login: 232,8. Suma de scripts iniciales, no bytes transferidos medidos ni imports posteriores. |
| Navegador | Marcado local de compra sobrevive recarga; navegación semanal sin URL actualizada reproducida; layouts y truncamientos descritos en sección 8. |
| RLS/datos | 7 tablas protegidas; 28 semanas completas; 108 listas; máximo 42 listas en una semana. Lectura remota únicamente. |

### Core Web Vitals

**No se obtuvieron LCP, CLS o INP válidos de campo, ni una corrida Lighthouse controlada.** No atribuyo una puntuación Lighthouse ni convierto tiempo del endpoint en LCP. La inspección señala:

- **LCP/contenido útil:** semana y creación usan `Suspense fallback={null}` y datos tras montaje; la caché puede adelantar contenido, pero el remoto llega tras import dinámico, Auth y dos consultas secuenciales. Un título visible pronto no garantiza que el menú útil esté listo.
- **CLS:** placeholders pequeños pasan a formularios/listas; categorías aparecen después de cargar. La lectura DOM inmediatamente tras recargar compra mostró el estado de carga antes de la lista. Hay riesgo de desplazamiento; no se ha cuantificado.
- **INP:** marcar productos cambia estado al instante y envía red después; no se observó bloqueo apreciable. Sin medición de eventos/dispositivo no puede afirmarse un buen percentil 75. El principal defecto del clic es lo que escribe, no el cálculo del contador.
- **Fuentes/imágenes:** `next/font` ya autoaloja fuentes. No hay fotos grandes ni `next/image` en los recorridos; optimizar imágenes no es la prioridad. Geist Mono está declarado sin uso de tipografía mono observado; comprobar carga real antes de quitarlo.

- **Navegación:** `prefetch={false}` en todos los enlaces inferiores elimina precarga. Medir antes de cambiar; puede ahorrar solicitudes, pero retrasa navegación en red lenta.

**Plan de medición:** Web Vitals por ruta/dispositivo y estado autenticado sin PII; p75 de LCP/CLS/INP, y tiempo separado hasta menú útil. Laboratorio en build de producción, conexión móvil simulada, caché fría/caliente y varios intentos. Comparar baseline tras reparar duplicación de compra, no optimizar contadores o `useMemo` por intuición.

#### Backend: mejoras medibles primero

1. Evitar la segunda compra por firma/fechas y recuperar compra existente en lugar de inferirla de nuevo.
2. Cambiar un check de O(n) ingredientes insertados + padre a una actualización de fila.
3. Eliminar espera de compra antes de mostrar una semana ya guardada; conservar estado de compra pendiente/reintento.
4. Precargar/coordinar identidad, perfil e historial; la creación consulta usuario → semanas → platos → INSERT telemetría antes de pedir IA, y espera UPDATE antes de mostrar propuesta. Telemetría no debería bloquear la vista previa.
5. Medir importación/compra con los mismos campos de tiempo, coste, intento y resultado que menú.

## 14. Deuda técnica y dependencias — lista corta

| Prioridad | Categoría | Deuda y tratamiento |
|---|---|---|
| 1 | DEUDA TÉCNICA | Esquema real no reproducible desde las tres migraciones versionadas; faltan creación inicial y varias evoluciones. Obtener baseline revisado, sin aplicarlo a producción como “reset”. |
| 2 | REFACTOR | `CreateMenuPage` 983 líneas y perfil 900: separar carga/contexto, acciones, presentación y persistencia después de testear el contrato. No una división puramente por tamaño. |
| 3 | DEUDA TÉCNICA | Tipos manuales de DB incorrectos, `goal/goals`, `userId/user_id`, tipos de producción en mocks y estados de slot dobles. Derivar tipos y mantener adaptadores de frontera. |
| 4 | REFACTOR | Unificar acceso a perfil/storage y errores de sync; normalizadores y categorías duplicados. Mantener funciones pequeñas para calendario/agrupación. |
| 5 | QUICK WIN | Retirar código sin consumidores cuando se confirme con build: `DayCard`, `mockImportedWeek`, catálogo mock no utilizado y shadcn card/badge/checkbox. Mantener textarea/button que sí se usan. |

| Prioridad | Categoría | Deuda y tratamiento |
|---|---|---|
| 6 | DEUDA TÉCNICA | README sigue siendo el de create-next-app. Añadir variables de ejemplo sin secretos, diagrama, scripts seguros, versión Node, instrucciones de DB/pruebas y significado de telemetría. |

**Dependencias: decisión selectiva**

- **Mantener:** Next/React/TypeScript/Tailwind, OpenAI, Supabase. No hay motivo para migrar de stack.
- **Mantener:** Radix Slot, class-variance-authority, clsx, tailwind-merge; se usan en componentes activos. La capa shadcn no es por sí sola un problema.
- **Candidata a retirar:** `@radix-ui/react-checkbox`; su único consumidor es el wrapper shadcn sin uso, mientras la lista usa checkbox nativo.
- **Mover o sustituir al ordenar scripts:** `dotenv` solo se usa en scripts; puede ser devDependency o sustituirse por carga nativa si se fija Node. No una mejora de runtime importante.
- **No eliminar `mockWeek.ts` de golpe:** contiene tipos que sí utiliza producción. Extraerlos primero.
- **Parchar selectivamente:** Next y tránsito runtime; `@supabase/auth-js` llega a través de supabase-js 2.49.4; avisos de Babel/humanfs/browserslist/js-yaml/brace-expansion etc. corresponden en gran parte a herramientas. No equivalen al mismo riesgo externo que el servidor.
- **No añadir dependencias por anticipación:** React Query, Zod, Zustand, date-fns o un framework de agentes pueden ser útiles, pero los defectos actuales se solucionan también con contratos y módulos pequeños. Un validador de esquemas sí puede justificarse si evita la deriva ya observada.
- No se encontraron any, `@ts-ignore` o `@ts-expect-error` explícitos en `src` durante el barrido. El problema de tipos son casts y contratos manuales incorrectos, no abuso generalizado de any.

# 15. Cosas que no cambiaría

1. Monolito Next con Supabase; apropiado para este producto y tamaño.
2. RLS basado en propietario y recorrido por padres; añadir tests, no reemplazarlo por service role desde navegador.
3. Identidad ISO de semana y unicidad usuario/año/semana; corregir bordes y transporte.
4. Generar → revisar → aplicar; preservar la posibilidad de decidir antes de guardar.
5. Elección compacta 7+7 de IA y construcción determinista de días/estados.
6. Timeouts/retries acotados y `store:false` del runtime nuevo de menú.
7. Referencias culinarias locales con procedencia y sin infraestructura de recuperación innecesaria.
8. Edición manual de platos y modo local de consulta; aislarlos y explicar sus límites.
9. Filas de compra grandes, navegación inferior, safe-area, componentes visuales y paleta cálida.
10. TypeScript estricto, React Compiler, lockfile y tests focalizados existentes. Ampliar protección donde faltan garantías.

# 16. Si empezásemos hoy desde cero

| Diferencia importante | Clasificación | Razón aplicada a beCook |
|---|---|---|
| El mismo monolito Next + Supabase | **Mantener como está** | No se necesita otro backend distribuido. |
| Auth/cuota/telemetría dentro del servidor que llama a IA | **Mejorar progresivamente** | Ahora el cliente controla la trazabilidad y los endpoints son públicos. |

| Diferencia importante | Clasificación | Razón aplicada a beCook |
|---|---|---|
| Fuente remota canónica y caché por usuario/semana/versionado | **Mejorar progresivamente** | Evita mezcla de cuentas y la ausencia en otro dispositivo. |
| Operaciones por entidad, transacciones para conjuntos | **Mejorar progresivamente** | Sustituye delete/insert y snapshots por clic. |
| Contrato compartido semana→ingredientes→compra | **Reemplazar en el futuro** la inferencia independiente | Hoy pierde raciones/restricciones y no tiene identidad de receta. |
| Perfil mínimo orientado a preferencias y restricciones | **Mejorar progresivamente** | Nacimiento/medidas no influyen en los nombres generados. |
| Calendario y parser de formato claro en código | **Mejorar progresivamente** | Ya hay helpers; se reutilizan antes de IA, con fechas reales. |
| Componentes de página con controladores y pruebas | **Mejorar progresivamente** | Extraer por responsabilidad cuando los flujos estén protegidos. |
| Biblioteca híbrida pequeña de recetas revisadas | **Reemplazar en el futuro** el catálogo mock muerto | Aporta valor si sirve a semana y compra, no como pantalla aislada. |
| UI móvil actual con contraste/foco/contexto corregidos | **Mantener la base y mejorar progresivamente** | No requiere un nuevo lenguaje visual ni PWA. |

# 17. Siguiente paso

**Primer paquete concreto para la siguiente sesión: “Protección de IA y guardado veraz”.**

- Preparar el tratamiento del archivo de recuperación, sin imprimir su contenido ni regenerar credenciales automáticamente.
- Verificar sesión en los tres endpoints de pago, transmitir token desde sus consumidores y mostrar una petición de login clara cuando falte sesión. Mantener consulta/edición local.
- Limitar cuerpos y campos de entrada; tests que demuestren que rechazos no llaman a OpenAI.
- Hacer que `saveWeekWithSupabaseSync` devuelva un resultado explícito y que «Usar este menú» diferencie local/sincronizado/compra pendiente/error, conservando la propuesta.
- Validar con fallos de red y usuario invitado, lint, tipos y build. Sin migraciones destructivas, sin limpiar datos existentes.

En un cambio pequeño separado, preparar el parche de Next y lockfile. Después abordar fechas/firma de compra y salsas vagas; a continuación identidad de caché y compra por semana. **Este informe no implementa ninguno de esos cambios.**